

Advanced Strategies for PDF Text Extraction and Data Cleaning in RAG Systems

Detailed Notes - Agentic AI Bootcamp

Contents

1	Introduction	2
2	Categorization of PDF Documents	2
3	Extraction Strategies for Text-Based PDFs	2
3.1	Standard Text Extraction	2
3.2	Layout-Aware Parsing	2
4	Extraction Strategies for Scanned PDFs (OCR)	2
4.1	Pre-processing Images	3
4.2	OCR Engines	3
5	Specialized Extraction Techniques	3
5.1	Table Extraction	3
5.2	Optical Mark Recognition (OMR)	3
5.3	Figures and Charts	3
6	Data Cleaning Post-Extraction	4
6.1	Artifact Removal	4
6.2	Text Normalization	4
6.3	Formatting and Metadata Enrichment	4
7	Conclusion	4

1 Introduction

In Retrieval-Augmented Generation (RAG) pipelines, the quality of the generated response is strictly upper-bounded by the quality of the retrieved context. PDFs remain the de facto standard for enterprise documents (reports, research papers, manuals), but their format is fundamentally presentation-oriented, not semantically structured. This document outlines advanced strategies for extracting, parsing, and cleaning text from various PDF types before the chunking stage.

2 Categorization of PDF Documents

Before applying extraction techniques, it is essential to classify the PDF into one of two primary categories:

- **Text-Based (Native) PDFs:** Generated directly from digital sources (e.g., MS Word, LaTeX). They contain character encodings and explicit text layers.
- **Scanned (Image-Based) PDFs:** Created via physical scanners. They contain no text layer, only rasterized images of pages.

3 Extraction Strategies for Text-Based PDFs

Native PDFs require libraries that can read the internal text streams.

3.1 Standard Text Extraction

- **PyMuPDF (fitz):** Highly performant, offers bounding box access, and preserves reading order reasonably well. Ideal for extracting text, images, and basic layout information.
- **pdfplumber:** Excellent for detailed character-level bounding box analysis. Slower than PyMuPDF but superior for debugging complex layouts and extracting tabular data.

3.2 Layout-Aware Parsing

Standard extraction often fails on multi-column layouts, interleaving text across columns.

- **Heuristic Methods:** Sorting text blocks by their (x, y) coordinates.
- **Machine Learning Models:** Utilizing layout analysis models (e.g., LayoutLMv3, YOLOX-based parsers like Unstructured) to classify blocks into **Title**, **Text**, **List**, **Table**, and **Figure**.

4 Extraction Strategies for Scanned PDFs (OCR)

For scanned documents, Optical Character Recognition (OCR) is mandatory.

4.1 Pre-processing Images

OCR accuracy drops significantly with poor image quality. Pre-processing steps include:

- **Binarization:** Converting grayscale/color images to black and white (e.g., Otsu's thresholding, adaptive thresholding).
- **Deskewing:** Rotating the image to correct scanning tilt.
- **Denoising:** Applying Gaussian blurs or median filters to remove background artifacts.

4.2 OCR Engines

- **Tesseract OCR:** Open-source, supports many languages. Requires rigorous image pre-processing for optimal results.
- **EasyOCR / PaddleOCR:** Deep learning-based, handles varied fonts, orientations, and complex backgrounds better than Tesseract.
- **Cloud APIs:** AWS Textract, Google Cloud Document AI, and Azure Document Intelligence provide state-of-the-art accuracy, including built-in layout and table understanding.

5 Specialized Extraction Techniques

5.1 Table Extraction

Tables contain dense relational data that standard text extraction flattens into unreadable strings.

- **Tools:** Camelot, Tabula, or OCR-based table parsers (Table Transformer).
- **Representation:** Convert extracted tables into Markdown, HTML, or CSV string formats before chunking to preserve row/column relationships for the LLM.

5.2 Optical Mark Recognition (OMR)

Used for forms with checkboxes or radio buttons.

- **Strategy:** Detect contour boxes using OpenCV. Calculate the pixel density (ratio of black to white pixels) inside the box. A density above a specific threshold implies a "checked" state.
- **Mapping:** Map the checked state to adjacent text to yield key-value pairs (e.g., `Is_Employed: True`).

5.3 Figures and Charts

- **Extraction:** Use PyMuPDF to extract images.
- **Vision-Language Models (VLMs):** Pass extracted images through models like LLaVA or GPT-4o to generate descriptive captions, injecting these captions back into the text stream.

6 Data Cleaning Post-Extraction

Raw extracted text is rarely ready for embedding and chunking. A robust cleaning pipeline is critical.

6.1 Artifact Removal

- **Headers/Footers & Page Numbers:** Often interrupt sentence flow across page boundaries. Remove using regex (e.g., `^\d+$`) or by ignoring text within the top/bottom 10% of the page bounding box.
- **Watermarks:** Filter out repetitive strings that have low opacity or span diagonally across bounding boxes.

6.2 Text Normalization

- **De-hyphenation:** Words split across lines (e.g., “docu-\nment”) must be rejoined. Use regex: `re.sub(r'-\n\s*', '', text)` (with dictionary checks to avoid merging legitimate hyphenated words).
- **Whitespace Normalization:** Collapse multiple spaces, tabs, and spurious newlines into single spaces.
- **Unicode Normalization:** Convert ligatures (e.g., “fi”, “fl”) to standard characters using NFKC normalization.

6.3 Formatting and Metadata Enrichment

- **Markdown Conversion:** Where possible, convert headings into Markdown (e.g., `# Heading`) to provide structural cues for chunking algorithms (like LangChain’s `MarkdownHeaderTextSplitter`).
- **Metadata Tagging:** Append source metadata (page number, document ID, section title) to the cleaned text blocks. This is vital for downstream Graph-RAG or precise citation generation.

7 Conclusion

Thorough PDF extraction and data cleaning are non-negotiable precursors to effective RAG. By treating native and scanned PDFs distinctly, deploying layout-aware parsing, utilizing OMR for forms, and rigorously normalizing the extracted text, we ensure that the subsequent chunking phase operates on coherent, high-fidelity semantic units.